

Examen Semestre 2

Matière : Algorithmique Et Structures de Données 2

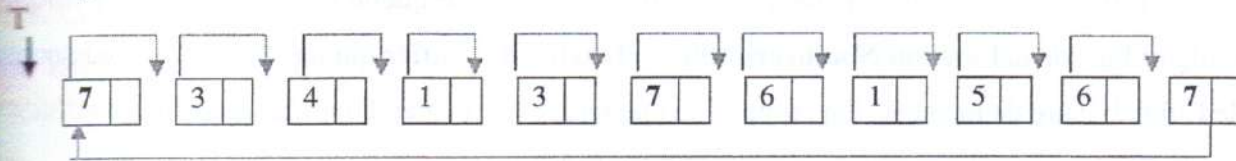
Enseignantes	H. Grira, R. Guedria	Date : Mai 2023
Niveau	1 ^{ère} Année	Durée : 01H30
Filière	LISI	Documents : non autorisés
Barème approximatif	5 pts + 6 pts + 4 pts + 5 pts	Nombre de pages : 2

Exercice 1 (5 points)

Soit T la tête d'une liste chaînée **circulaire** d'entiers quelconques.

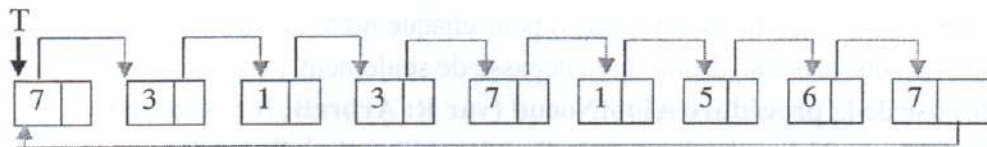
Ecrire l'algorithme de la **procédure DeleteBefore(var T :Pliste; X : entier)** qui permet de supprimer l'élément qui se trouve juste avant chaque X et qui est différent de X.

Exemple :



Pour X = 1, la procédure supprime les valeurs 4 et 6 (4 avant 1 et 6 avant 1)

Après appel à la procédure **DeleteBefore**, la liste Contient :



Exercice 2 File (6 points):

- Ecrire l'algorithme de la **fonction Recherche (Var F : File d'entiers; X : entier) : booléen** qui retourne Vrai si X se trouve dans la file F. (Le contenu de F ne doit pas changer)
- Ecrire l'algorithme de la **procédure NearLike (Var F : file d'entiers; X : entier)** qui permet d'insérer X à coté d'un élément qui lui est égal ou à la fin si aucun élément n'est égal à X.

Exemple :

F → 23 16 17 6 11 4 42 35 18

X = 11

Après appel à la procédure **NearLike** la file contient :

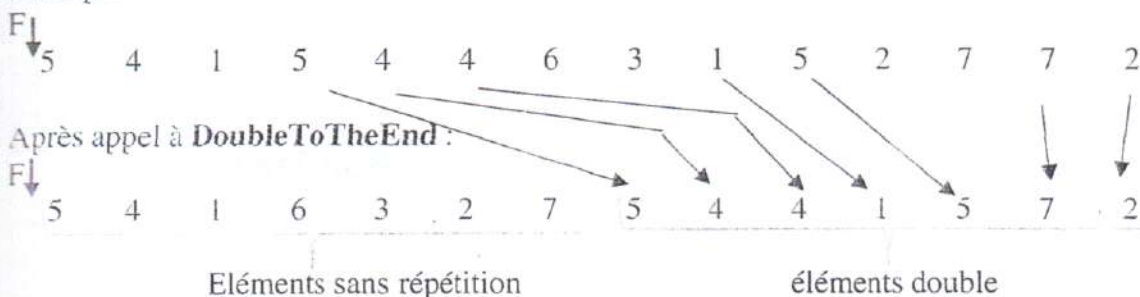
F → 23 16 17 6 **11 11** 4 42 35 18 (on a ajouté X à coté de 11)

Si X = 29, après appel à la procédure **NearLike**, la file contient :

F → 23 16 17 6 11 4 42 35 18 **29** (on a ajouté X à la fin de F)

- Ecrire l'algorithme de la **procédure DoubleToTheEnd(var F : File d'entier)** qui permet de déplacer les valeurs en double vers la fin de la file.

Exemple



On suppose qu'on dispose des primitives **Enfiler(F, X)** ; **Défiler(F)** et **FileVide(F)**

Exercice 3 Arbre Binaire de Recherche (5 points)

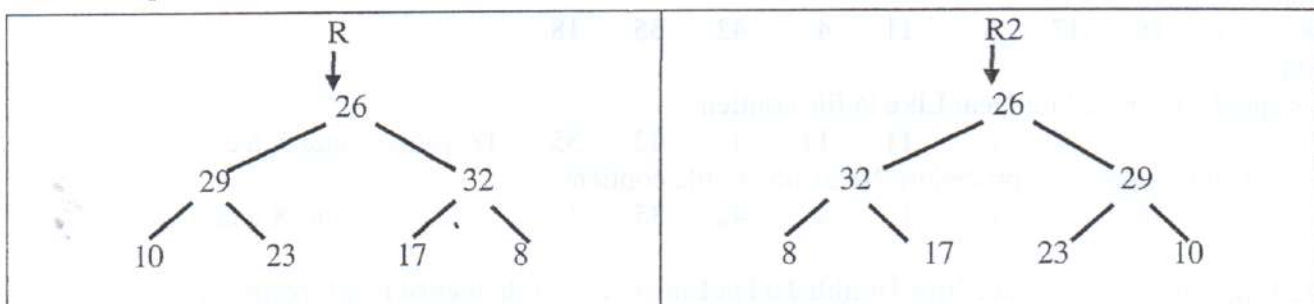
1. Dessiner l'Arbre binaire de recherche obtenu en insérant successivement les éléments suivants et selon cet ordre : 19, 10, 27, 7, 3, 9, 17, 15, 22, 35, 30, 25, 20
2. Donner l'ordre des nœuds visités selon le parcours Infixe (Droite - Père - Gauche).
3. Décrire brièvement le principe de suppression d'un nœud dans un Arbre binaire de recherche ABR en traitant tous les cas possibles pour le nœud à supprimer. (sans donner l'algorithme).
4. Dessiner l'ABR obtenu après la suppression successive des nœuds 30, 10 et 17 en précisant chaque fois le cas et la décision prise (si le nœud a été remplacé, préciser par quelle valeur et pourquoi).
5. Ecrire l'algorithme de la **fonction NombreFeuilles (R:ArbreB) : entier** qui retourne le nombre de feuilles de l'arbre binaire de racine R. On rappelle qu'une feuille est un nœud qui n'a aucun fils.

Exercice 4 Arbre Binaire (4 points)

Un arbre binaire de racine R est dit en équilibre si pour chaque nœud, la taille du sous arbre de gauche est égale à la taille du sous arbre de droite ou la dépasse de seulement 1 ($tg=td$ ou $tg = td + 1$). On suppose qu'on dispose de la **procédure AjoutNoeud (var R: ArbreB; N : ArbreB)** qui permet d'ajouter le nœud d'adresse N dans le premier emplacement libre dans l'arbre R en veillant à avoir un arbre équilibré.

Ecrire l'algorithme de la **procédure Miroir (R, var R2 : ArbreB)** qui permet de créer l'arbre binaire de racine R2 contenant les mêmes valeurs de R tel que pour chaque nœud, les valeurs de gauche sont échangées avec les valeurs de droite (vous pouvez faire appel à **AjoutNoeud**).

Exemple :



Bon Travail

